

- 《程序分析》实验报告
- 实验六 上下文敏感的指针分析
 - 一、实验目标
 - 二、实验理论
 - 2.1 指针分析与 points-to 集合
 - 2.2 上下文敏感指针分析
 - 2.3 新增分析规则
 - 2.4 上下文敏感策略
 - 三、实验实现
 - 3.1 Solver.addReachable 实现
 - 3.2 StmtProcessor 语句处理
 - 3.3 addPFGEEdge 与差分传播
 - 3.4 analyze 主循环实现
 - 3.5 processCall 实例方法调用处理
 - 3.6 上下文选择器实现
 - 3.6.1 一层调用点敏感
 - 3.6.2 两层调用点敏感
 - 3.6.3 一层和两层对象敏感
 - 3.6.4 一层和两层类型敏感
 - 四、实验结果
 - 4.1 测试用例覆盖
 - 4.2 静态字段结果
 - 4.3 数组索引结果
 - 4.4 对象敏感精度结果
 - 4.5 两层对象敏感结果
 - 4.6 提交内容
 - 五、实验总结
 - 参考资料

《程序分析》实验报告

学院：软件工程学院

专业：软件工程

姓名：甘志熙

学号：23331040

日期：2026 年 6 月 11 日

实验六 上下文敏感的指针分析

一、实验目标

1. 在 Tai-e 静态分析框架中实现 Java 程序的上下文敏感指针分析框架。
2. 在指针分析过程中同步构建上下文敏感调用图，使调用图边能够随着 points-to 信息的增长逐步发现。
3. 在上下文敏感指针分析中支持局部变量、实例字段、静态字段、数组索引四类指针，以及实例方法和静态方法调用。
4. 实现三类常见上下文敏感策略：调用点敏感、对象敏感和类型敏感，并分别完成一层与两层上下文选择器。
5. 通过 Tai-e 提供的 `CSPTATest` 测试用例检查实现的正确性，观察上下文敏感分析相较于非上下文敏感分析在精度上的提升。

二、实验理论

2.1 指针分析与 points-to 集合

指针分析用于计算程序中每个指针变量可能指向的抽象对象集合。对 Java 程序而言，指针不仅包括局部变量，还包括对象字段、静态字段和数组元素。Tai-e 中用 `PointsToSet` 表示某个指针可能指向的对象集合，用 `CSObj` 表示带上下文的抽象对象。

本实验中的指针分析以指针流图（Pointer Flow Graph, PFG）为核心。PFG 的节点是各种指针，边表示对象可能从一个指针流向另一个指针。例如赋值语句 `x = y` 会产生一条从 `y` 到 `x` 的 PFG 边；字段读写和数组读写则会在对象被发现之后动态产生新的边。

2.2 上下文敏感指针分析

非上下文敏感指针分析会把同一个方法的所有调用混在一起分析，因此容易把不同调用路径下的对象合并到同一个 points-to 集合中。上下文敏感分析为方法、变量、调用点和对象附加上下文，从而区分同一方法在不同调用环境下的分析结果。

Tai-e 中与上下文相关的核心元素包括：

- `Context`：表示上下文。

- **CSVar**：带上下文的变量。
- **CSObj**：带上下文的抽象对象。
- **CSCallSite**：带上下文的调用点。
- **CSMethod**：带上下文的方法。
- **CSManager**：统一创建和管理上述上下文敏感元素。

在上下文敏感指针分析中，调用图不再预先由 CHA 一次性构建，而是随着接收者变量 `points-to` 集合的增长动态解析调用目标。每当某个 receiver 变量发现新的对象，分析器就可以根据该对象的实际类型解析实例调用目标，并在调用图中加入新的上下文敏感调用边。

2.3 新增分析规则

本实验除课堂中介绍的局部变量、实例字段和实例方法调用外，还需要处理静态字段、数组索引和静态方法。

静态字段的处理不依赖具体对象。对于 `T.f = y`，只需要加入从变量 `y` 到静态字段指针 `T.f` 的 PFG 边；对于 `y = T.f`，加入从静态字段指针 `T.f` 到变量 `y` 的 PFG 边。

数组索引采用数组汇总抽象，不区分同一个数组对象的不同下标。对于数组对象 `o`，用 `o[*]` 表示该数组所有元素位置的统一抽象指针。`x[i] = y` 会产生从 `y` 到 `x` 指向数组对象的 `[*]` 指针的边；`y = x[i]` 会产生从 `x` 指向数组对象的 `[*]` 指针到 `y` 的边。

静态方法调用不需要 receiver object，也不需要动态分派。处理静态调用时，只需要直接解析被调方法，选择被调方法上下文，建立调用图边，并传播实参和返回值。

2.4 上下文敏感策略

本实验实现三类上下文敏感策略：

- 调用点敏感 (call-site sensitivity)：使用调用点作为上下文元素。
- 对象敏感 (object sensitivity)：使用 receiver 对象作为上下文元素。
- 类型敏感 (type sensitivity)：使用 receiver 对象所在的类型作为上下文元素。

对于一层上下文敏感策略，方法上下文只保留一个上下文元素，堆上下文为空。对于两层上下文敏感策略，方法上下文保留两个上下文元素，堆上下文保留一层，即通常使用方法上下文中的最后一个元素作为新建对象的堆上下文。

三、实验实现

本实验主要完成以下 7 个类中的 TODO 部分：

```
Solver.java
_1CallSelector.java
_1ObjSelector.java
_1TypeSelector.java
_2CallSelector.java
_2ObjSelector.java
_2TypeSelector.java
```

3.1 Solver.addReachable 实现

`addReachable` 用于处理新发现的上下文敏感方法。实现时先调用 `callGraph.addReachableMethod(csMethod)` 做去重，避免重复扫描同一个上下文敏感方法；随后跳过抽象方法，并使用 `StmtProcessor` 遍历该方法 IR 中的所有语句。

```
private void addReachable(CSMethod csMethod) {
    if (!callGraph.addReachableMethod(csMethod)) {
        return;
    }
    JMethod method = csMethod.getMethod();
    if (method.isAbstract()) {
        return;
    }
    StmtProcessor processor = new StmtProcessor(csMethod);
    method.getIR().forEach(stmt -> stmt.accept(processor));
}
```

该方法的作用是把新可达方法内部的初始约束加入分析系统。例如 `new` 语句会产生初始 `points-to` 信息，赋值语句会产生 PFG 边，静态字段和静态方法调用也会在这一阶段被处理。

3.2 StmtProcessor 语句处理

在 `New` 语句中，先通过 `heapModel.getObj(stmt)` 得到抽象对象，再调用 `contextSelector.selectHeapContext(csMethod, obj)` 选择堆上下文，最后将带上下文对象加入左值变量的 `points-to` 集合传播队列。

```

public Void visit(New stmt) {
    Obj obj = heapModel.getObj(stmt);
    Context heapContext = contextSelector.selectHeapContext(csMethod, obj);
    CSObj csObj = csManager.getCSObj(heapContext, obj);
    workList.addEntry(csManager.getCSVar(context, stmt.getLValue()),
        PointsToSetFactory.make(csObj));
    return null;
}

```

在 **Copy** 语句中， $x = y$ 被转化为从右值变量到左值变量的 PFG 边：

```

public Void visit(Copy stmt) {
    addPFGEEdge(csManager.getCSVar(context, stmt.getRValue()),
        csManager.getCSVar(context, stmt.getLValue()));
    return null;
}

```

静态字段不依赖 receiver 对象，因此可以在扫描语句时直接处理。静态字段 store 建立 变量 $\rightarrow$ 静态字段 的边，静态字段 load 建立 静态字段 $\rightarrow$ 变量 的边。

```

public Void visit(StoreField stmt) {
    if (stmt.isStatic()) {
        JField field = stmt.getFieldRef().resolve();
        addPFGEEdge(csManager.getCSVar(context, stmt.getRValue()),
            csManager.getStaticField(field));
    }
    return null;
}

public Void visit(LoadField stmt) {
    if (stmt.isStatic()) {
        JField field = stmt.getFieldRef().resolve();
        addPFGEEdge(csManager.getStaticField(field),
            csManager.getCSVar(context, stmt.getLValue()));
    }
    return null;
}

```

静态方法调用在扫描阶段即可解析目标方法。实现中先调用 `resolveCallee(null, stmt)` 得到被调方法，再用 `contextSelector.selectContext(csCallSite, callee)` 选择被调方法上下文。加入调用图边后，继续处理实参到形参、返回值到调用结果的 PFG 边。

```

public Void visit(Invoke stmt) {
    if (!stmt.isStatic()) {
        return null;
    }
    JMethod callee = resolveCallee(null, stmt);
    CSCALLSite csCallSite = csManager.getCSCALLSite(context, stmt);
    Context calleeContext = contextSelector.selectContext(csCallSite, callee);
    CSMETHOD csCallee = csManager.getCSMETHOD(calleeContext, callee);
    Edge<CSCALLSite, CSMETHOD> edge = new Edge<>(
        CallGraphs.getCallKind(stmt), csCallSite, csCallee);
    if (callGraph.addEdge(edge)) {
        addReachable(csCallee);
    }
    ...
    return null;
}

```

3.3 addPFGEdge 与差分传播

addPFGEdge 对应算法中的 AddEdge。它先尝试向 PFG 中加入新边，如果该边是新边，并且源指针当前 points-to 集不为空，就立即把源指针已有对象加入目标指针的 worklist 条目中。

```

private void addPFGEdge(Pointer source, Pointer target) {
    if (pointerFlowGraph.addEdge(source, target)
        && !source.getPointsToSet().isEmpty()) {
        workList.addEntry(target, source.getPointsToSet());
    }
}

```

这样做可以保证后加入的 PFG 边不会遗漏已经发现的 points-to 信息。例如某个字段边在对象发现之后才被加入，如果不立即传播源节点已有集合，就可能丢失之前已经计算出的对象。

propagate 负责计算差分集合。它只把真正新增的对象加入 **delta**，并将 **delta** 继续传播到 PFG 后继节点。

```

private PointsToSet propagate(Pointer pointer, PointsToSet pointsToSet) {
    PointsToSet delta = PointsToSetFactory.make();
    for (CSObj obj : pointsToSet) {
        if (pointer.getPointsToSet().addObject(obj)) {
            delta.addObject(obj);
        }
    }
}

```

```

    if (!delta.isEmpty()) {
        for (Pointer succ : pointerFlowGraph.getSuccsOf(pointer)) {
            workList.addEntry(succ, delta);
        }
    }
    return delta;
}

```

该实现避免对已经传播过的对象重复触发下游事件，是 worklist 算法能够收敛的关键。

3.4 analyze 主循环实现

analyze 是求解器主循环。每次从 worklist 中取出一个条目，调用 **propagate** 更新指针的 points-to 集合。如果当前指针是 **CSVar** 且存在新增对象，则进一步触发实例字段、数组元素和实例方法调用的处理。

```

private void analyze() {
    while (!workList.isEmpty()) {
        WorkList.Entry entry = workList.pollEntry();
        Pointer pointer = entry.pointer();
        PointsToSet delta = propagate(pointer, entry.pointsToSet());
        if (!(pointer instanceof CSVar csVar) || delta.isEmpty()) {
            continue;
        }
        Var var = csVar.getVar();
        Context context = csVar.getContext();
        for (CSObj obj : delta) {
            ...
            processCall(csVar, obj);
        }
    }
}

```

实例字段 store/load 的处理依赖新发现的接收者对象。例如当变量 **x** 新指向对象 **o** 时，**x.f = y** 才能被具体化为从 **y** 到 **o.f** 的 PFG 边；**y = x.f** 才能被具体化为从 **o.f** 到 **y** 的 PFG 边。

数组 store/load 的处理方式与实例字段类似，只是字段指针换成数组汇总指针 **ArrayIndex**：

```

for (StoreArray store : var.getStoreArrays()) {
    addPFGEdge(csManager.getCSVar(context, store.getRValue()),
               csManager.getArrayIndex(obj));
}
for (LoadArray load : var.getLoadArrays()) {

```

```
addPFGEdge(csManager.getArrayIndex(obj),  
           csManager.getCSVar(context, load.getLValue()));  
}
```

3.5 processCall 实例方法调用处理

实例调用需要等 receiver 变量出现新对象之后才能解析。实现中遍历 receiver 变量关联的所有调用点，用 `resolveCallee(recvObj, callSite)` 根据接收者对象实际类型解析目标方法，再由 context selector 选择被调方法上下文。

```
private void processCall(CSVar recv, CSObj recvObj) {  
    Var recvVar = recv.getVar();  
    Context callerContext = recv.getContext();  
    for (Invoke callSite : recvVar.getInvokes()) {  
        JMethod callee = resolveCallee(recvObj, callSite);  
        CSCallSite csCallSite = csManager.getCSCallSite(callerContext, callSite);  
        Context calleeContext = contextSelector.selectContext(  
            csCallSite, recvObj, callee);  
        CSMethod csCallee = csManager.getCSMethod(calleeContext, callee);  
        ...  
    }  
}
```

随后需要完成三类传播关系：

1. 将 receiver 对象传给被调方法的 `this` 变量。
2. 将调用点实参传给被调方法形参。
3. 将被调方法返回变量传给调用点左值。

```
Var thisVar = callee.getIR().getThis();  
if (thisVar != null) {  
    workList.addEntry(csManager.getCSVar(calleeContext, thisVar),  
                     PointsToSetFactory.make(recvObj));  
}  
  
InvokeExp invokeExp = callSite.getInvokeExp();  
for (int i = 0; i < invokeExp.getArgCount(); ++i) {  
    addPFGEdge(csManager.getCSVar(callerContext, invokeExp.getArg(i)),  
              csManager.getCSVar(calleeContext, callee.getIR().getParam(i)));  
}  
  
if (callSite.getResult() != null) {  
    for (Var ret : callee.getIR().getReturnVars()) {  
        addPFGEdge(csManager.getCSVar(calleeContext, ret),  
                  csManager.getCSVar(callerContext, callSite.getResult()));  
    }  
}
```



```
}  
}
```

这部分体现了指针分析和调用图构建的互相依赖：points-to 信息帮助解析调用目标，新的调用目标又会带来新的可达方法和新的 PFG 边。

3.6 上下文选择器实现

3.6.1 一层调用点敏感

一层调用点敏感使用当前调用点作为被调方法上下文，堆上下文为空：

```
public Context selectContext(CSCallSite callSite, JMethod callee) {  
    return ListContext.make(callSite.getCallSite());  
}  
  
public Context selectContext(CSCallSite callSite, CSObj recv, JMethod callee) {  
    return ListContext.make(callSite.getCallSite());  
}  
  
public Context selectHeapContext(CSMethod method, Obj obj) {  
    return getEmptyContext();  
}
```

3.6.2 两层调用点敏感

两层调用点敏感保留调用者上下文最后一个调用点和当前调用点。若调用者上下文为空，则只使用当前调用点。

```
private Context append(Context context, Invoke callSite) {  
    return context.getLength() == 0  
        ? ListContext.make(callSite)  
        : ListContext.make(context.getElementAt(context.getLength() - 1),  
                           callSite);  
}
```

堆上下文保留方法上下文最后一层元素：

```
public Context selectHeapContext(CSMethod method, Obj obj) {  
    Context context = method.getContext();  
    return context.getLength() == 0  
        ? getEmptyContext()
```

```
        : ListContext.make(context.getElementAt(context.getLength() - 1));  
    }
```

3.6.3 一层和两层对象敏感

对象敏感策略对实例方法使用 receiver 对象作为上下文元素。静态方法没有 receiver，因此直接沿用调用者上下文。

```
public Context selectContext(CSCallSite callSite, JMethod callee) {  
    return callSite.getContext();  
}  
  
public Context selectContext(CSCallSite callSite, CSObj recv, JMethod callee) {  
    return ListContext.make(recv.getObject());  
}
```

两层对象敏感在 receiver 对象原有上下文后追加 receiver 对象本身，并截断为两层：

```
private Context append(Context context, Obj obj) {  
    return context.getLength() == 0  
        ? ListContext.make(obj)  
        : ListContext.make(context.getElementAt(context.getLength() - 1),  
                           obj);  
}
```

3.6.4 一层和两层类型敏感

类型敏感与对象敏感类似，但上下文元素使用 receiver 对象的 container type，而不是对象本身。

```
public Context selectContext(CSCallSite callSite, CSObj recv, JMethod callee) {  
    return ListContext.make(recv.getObject().getContainerType());  
}
```

两层类型敏感保留上一层类型上下文和当前 receiver 对象的 container type：

```
private Context append(Context context, Type type) {  
    return context.getLength() == 0  
        ? ListContext.make(type)  
        : ListContext.make(context.getElementAt(context.getLength() - 1),  
                           type);  
}
```

```
    type);  
}
```

这种实现能够区分来自不同类型上下文的调用，同时比对象敏感更粗粒度，通常在精度和开销之间取得折中。

四、实验结果

4.1 测试用例覆盖

Tai-e 为本实验提供了测试驱动类 `pascal.taie.analysis.pta.CSPTATest`。该测试类覆盖以下 13 组用例：

```
New  
Assign  
StoreLoad  
Call  
InstanceField  
OneCall  
OneObject  
OneType  
TwoCall  
TwoObject  
TwoType  
StaticField  
Array
```

这些用例分别覆盖对象创建、变量赋值、字段读写、方法调用、实例字段、调用点敏感、对象敏感、类型敏感、两层上下文、静态字段和数组索引等功能点。实验代码完成后，可通过 `CSPTATest` 与 `src/test/resources/pta/cspta/` 目录下的 `expected` 文件进行比对。

4.2 静态字段结果

`StaticField` 用例用于验证静态字段 store/load 的传播。期望结果中，静态字段 `<A: B b>` 正确指向在 `main` 方法第 4 行创建的 `B` 对象：

```
Points-to sets of all static fields
```

```
<A: B b> -> [[]:NewObj{<StaticField: void main(java.lang.String[])>[0@L4] new B}]
```

这说明实现中对 $T.f = y$ 和 $y = T.f$ 的 PFG 边添加是正确的，静态字段能够作为独立指针参与 points-to 传播。

4.3 数组索引结果

Array 用例用于验证数组 store/load 的处理。期望结果中，同一个数组对象的不同下标被汇总为 **[*]** 指针。例如 **A[]** 数组对象的数组索引指针指向两个 **A** 对象：

```
Points-to sets of all array indexes
```

```
[]:NewObj{<Array: void main(java.lang.String[])>[1@L4] newarray A[%intconst0]}[*]  
-> [[]:NewObj{<Array: void main(java.lang.String[])>[2@L5] new A},  
    []:NewObj{<Array: void main(java.lang.String[])>[6@L6] new A}]
```

B[] 数组对象的数组索引指针指向通过 **arrayStore** 存入的 **A** 对象：

```
[]:NewObj{<Array: void main(java.lang.String[])>[13@L9] newarray B[%intconst0]}[*]  
-> [[]:NewObj{<Array: void main(java.lang.String[])>[14@L10] new A}]
```

这说明数组读写不是在扫描语句时静态加入固定边，而是在 receiver 数组变量发现具体数组对象后，针对该对象的 **ArrayIndex** 指针动态建立 PFG 边。

4.4 对象敏感精度结果

OneObject 用例体现了一层对象敏感分析对精度的提升。非上下文敏感分析中，两个 **A** 对象的字段 **f** 会混合两个 **B** 对象；对象敏感分析能够区分不同 receiver 对象，使两个字段分别指向对应的 **B** 对象：

```
[]:NewObj{<OneObject: void m()>[0@L7] new A}.f  
-> [[]:NewObj{<OneObject: void m()>[6@L9] new B}]
```

```
[]:NewObj{<OneObject: void m()>[3@L8] new A}.f  
-> [[]:NewObj{<OneObject: void m()>[9@L10] new B}]
```

同时，方法上下文中可以看到 receiver 对象被用作上下文。例如 `A.get()` 在不同 `A` 对象上下文中拥有不同的 `temp$0 points-to` 结果：

```
[NewObj{<OneObject: void m()>[0@L7] new A}]:<A: B get()>/temp$0  
-> [[]:NewObj{<OneObject: void m()>[6@L9] new B}]  
  
[NewObj{<OneObject: void m()>[3@L8] new A}]:<A: void doSet(B)>/p  
-> [[]:NewObj{<OneObject: void m()>[9@L10] new B}]
```

这表明对象敏感策略成功区分了不同对象接收者下的方法调用，避免了不必要的 `points-to` 合并。

4.5 两层对象敏感结果

`TwoObject` 用例用于验证两层对象敏感策略。期望结果中，迭代器对象的上下文同时包含外层 `List` 对象和内部创建的 `ListIterator` 对象，例如：

```
[NewObj{<TwoObject: void m()>[0@L11] new List},  
NewObj{<List: Iterator iterator()>[0@L31] new List$ListIterator}]  
:<List$ListIterator: java.lang.Object next()>/temp$1  
-> [[]:NewObj{<TwoObject: void m()>[3@L12] new java.lang.Object}]
```

对于另一个 `List` 对象，上下文不同，因此 `next()` 返回的对象也能被区分：

```
[NewObj{<TwoObject: void m()>[6@L13] new List},  
NewObj{<List: Iterator iterator()>[0@L31] new List$ListIterator}]  
:<List$ListIterator: java.lang.Object next()>/temp$1  
-> [[]:NewObj{<TwoObject: void m()>[9@L14] new java.lang.Object}]
```

这说明两层对象敏感策略不仅区分当前 receiver，还能利用上一层对象上下文保留更长的调用历史，从而进一步提升精度。

4.6 提交内容

本实验最终提交的 zip 文件应包含以下 7 个实现类：

```
Solver.java  
_1CallSelector.java
```

```
_1ObjSelector.java  
_1TypeSelector.java  
_2CallSelector.java  
_2ObjSelector.java  
_2TypeSelector.java
```

作业要求中还需要提交实验报告和 OJ 评分截图。Markdown 报告可在最终整理时转换为 Word 或 PDF，并将 OJ 截图附在本节末尾。

五、实验总结

通过本次实验，我进一步理解了指针分析、调用图构建和上下文敏感策略之间的关系。与前一次非上下文敏感指针分析相比，本次实验不再把所有调用混在同一个方法实例中，而是通过 **Context** 为方法、变量、对象和调用点增加上下文信息，使分析结果能够区分不同调用环境。

在实现 **Solver** 的过程中，我认识到上下文敏感指针分析的核心是“按需发现约束”。对于 **new**、**copy**、静态字段和静态调用，可以在方法首次可达时直接处理；而实例字段、数组元素和实例调用依赖 receiver 变量的 points-to 信息，必须在 worklist 主循环中随着新对象的发现动态处理。这种设计使调用图构建和 points-to 传播互相促进：新的 points-to 信息发现新的调用边，新的调用边又引入新的方法和新的传播约束。

在实现 context selector 的过程中，我对调用点敏感、对象敏感和类型敏感的差异有了更清晰的理解。调用点敏感强调调用位置，对静态方法也能自然使用调用点作为上下文；对象敏感强调 receiver 对象，更适合面向对象程序中同一方法被不同对象调用的场景；类型敏感使用类型作为上下文，精度低于对象敏感，但上下文数量更少。两层策略相比一层策略能够保留更长的调用历史，但也会带来更多上下文敏感方法和变量实例。

本次实验的不足在于，目前实现主要关注正确性，没有对效率做进一步优化。例如 **addPFGEdge**、**processCall** 和 selector 的上下文构造都可以结合缓存进一步减少重复对象创建；在更大的程序上，PFG 边数、上下文数量和 worklist 条目数都可能快速增长。后续如果继续改进，可以考虑对上下文创建、调用解析结果和字段/数组边处理进行缓存，以降低分析开销。

参考资料

1. Tai-e 作业 6：上下文敏感的指针分析，<https://tai-e.pascal-lab.net/pa6.html>
2. **Week 13-实验六-上下文敏感的指针分析1班 (1).pdf**

